## 10 Install and Explore Selenium for automated testin

**What is Selenium?**

**Selenium is an open-source tool used for automating web browsers. It provides a suite of tools for testing web applications across different browsers and platforms. The Selenium WebDriver API allows you to interact with web pages programmatically by simulating user actions like clicking, typing, navigating, etc.**

---

**Installation of Selenium WebDriver**

**Step 1: Install Node.js and npm**

```
D:\py54\Java_test\Java_test>npm -v
10.2.5
```

**If you don't have Node.js installed, download it from Node.js official website. Follow the instructions for your operating system.**

1. **Verify the installation by running the following commands in the terminal:**

**node -v**

**npm -v**

```
D:\py54\Java_test\Java_test>node -v
v20.10.0
```

```
D:\>mkdir jtest

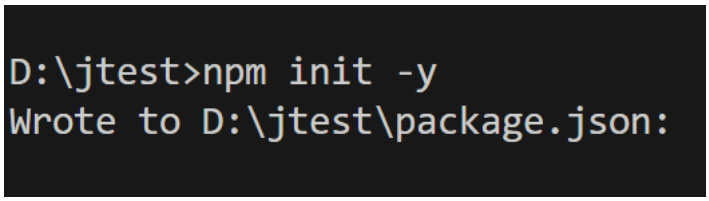
D:\>cd jtest
```

**Step 2: Create a Project Folder**

1. **Open Visual Studio Code.**
2. **Create a new folder named SeleniumLab.**

3. **Open the terminal in VS Code and initialize a Node.js project:**

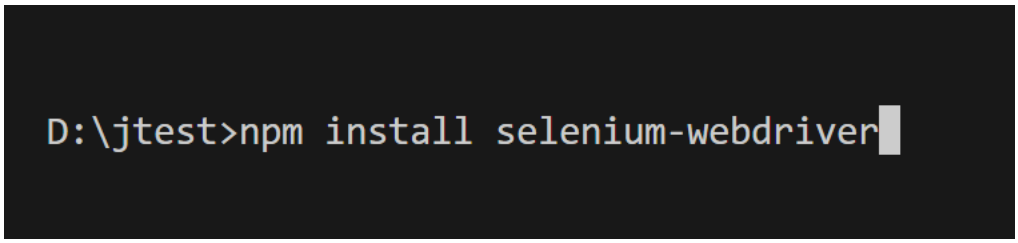
**npm init -y**



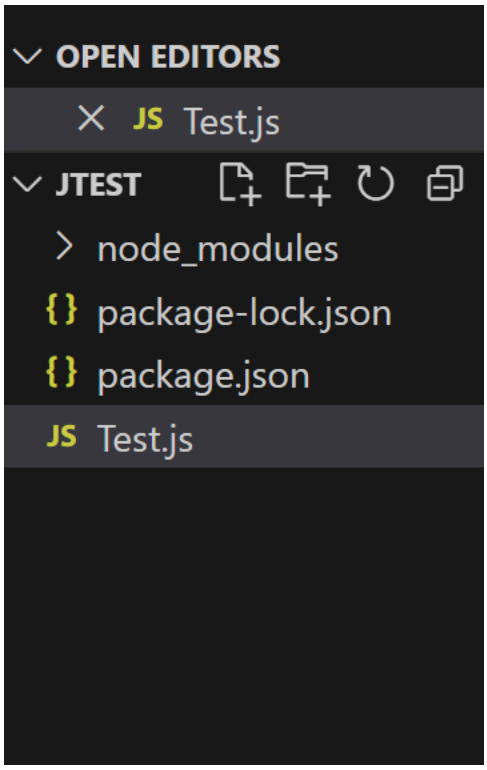

**Step 3: Install Selenium WebDriver**

1. **Install the selenium-webdriver package:**

**npm install selenium-webdriver**




2. **Download the ChromeDriver that matches your Chrome browser version from ChromeDriver Downloads.**
3. **Extract the downloaded ChromeDriver and place it in the project folder.**



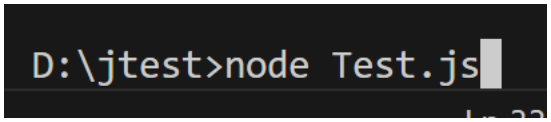

---

**Part 3: Writing a Simple Selenium Test Script**

**Step 1: Create a JavaScript Test File**

1. **Inside the SeleniumLab folder, create a new file named test.js.**
2. **Copy and paste the following code into test.js:**

**Test.js**




```
const { Builder, By, Key, until } = require('selenium-webdriver');

async function testGoogleSearch() {
   // Step 1: Launch the Chrome browser
   let driver = await new Builder().forBrowser('chrome').build();
   try {
      // Step 2: Open the Google homepage
```

```
        await driver.get('https://www.google.com');

        // Step 3: Locate the search box and enter a search query
        await driver.findElement(By.name('q')).sendKeys('Selenium WebDriver', Key.RETURN);

        // Step 4: Wait for the search results to load
        await driver.wait(until.titleContains('Selenium WebDriver'), 5000);

        // Step 5: Print the page title in the console
        let title = await driver.getTitle();
        console.log('Page Title:', title);

        // Step 6: Check if the search results are displayed
        if (title.includes('Selenium WebDriver')) {
            console.log('Test Passed: Search results loaded successfully.');
        } else {
            console.log('Test Failed: Search results not loaded.');
        }
    } catch (error) {
        console.error('Error:', error);
    } finally {
        // Step 7: Close the browser
        await driver.quit();
    }
}
```

**testGoogleSearch();**

**Explanation of the Code:**

1. **Import Selenium WebDriver:**
    - **Builder: Used to create a new browser instance.**
    - **By: Used to locate elements on the web page.**
    - **Key: Simulates keyboard actions.**
    - **until: Used for waiting until a condition is met.**
2. **testGoogleSearch Function:**
    - **Step 1: Launches the Chrome browser.**
    - **Step 2: Opens https://www.google.com.**
    - **Step 3: Finds the search box using its name attribute and types a query (Selenium WebDriver).**
    - **Step 4: Waits until the page title includes "Selenium WebDriver".**
    - **Step 5: Prints the page title.**
    - **Step 6: Verifies if the search results are displayed correctly.**
    - **Step 7: Closes the browser.**

---

**Part 4: Running the Selenium Test Script**

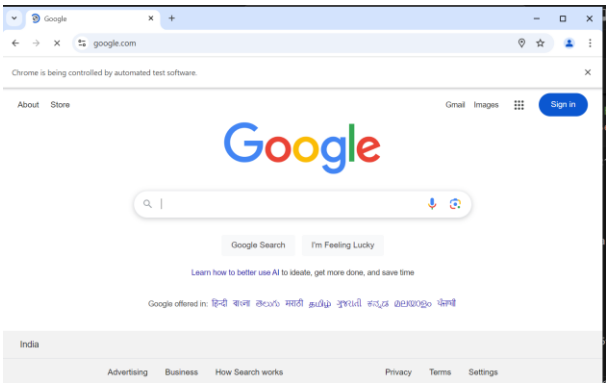
**Step 1: Running the Script**

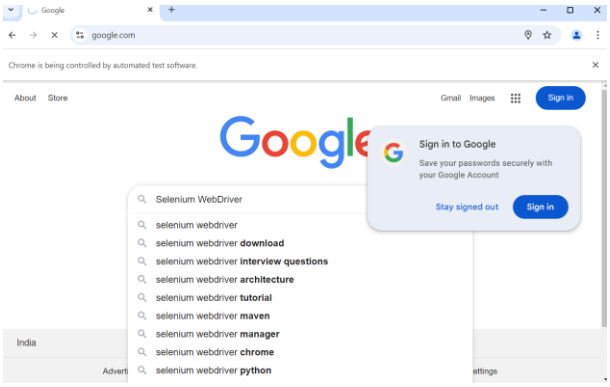
1. **Make sure that ChromeDriver is in the project folder.**
2. **Open the terminal in Visual Studio Code.**
3. **Run the test script:**

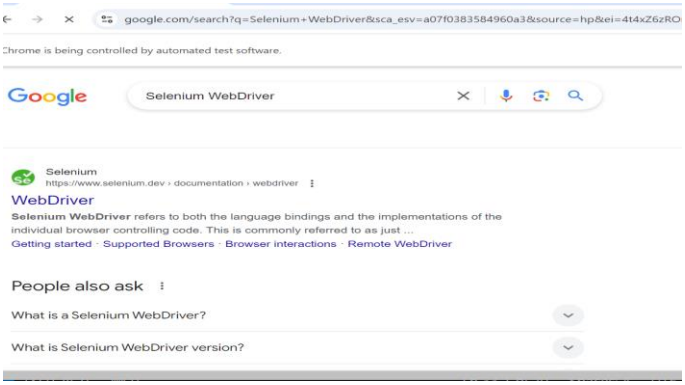
**node Test.js**

**Expected Output:**

- **The Chrome browser should open, navigate to Google, perform a search for "Selenium WebDriver", and display the search results.**
- **The terminal should display:**

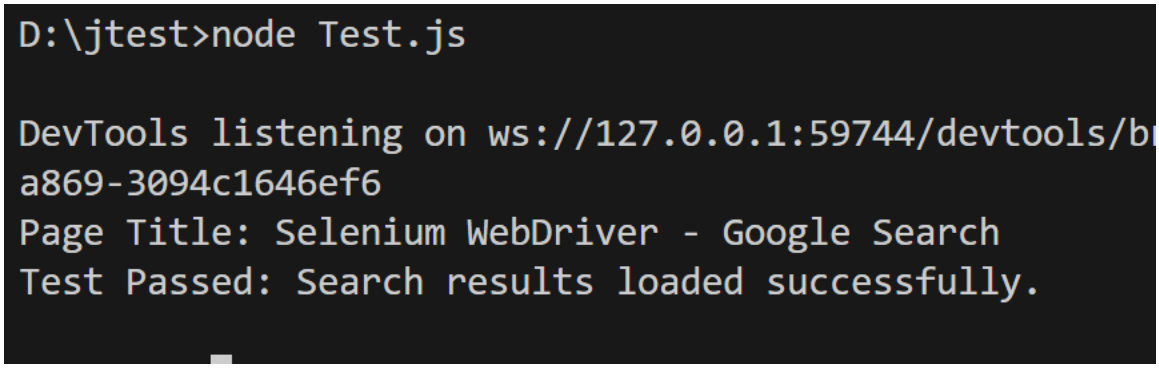



**Page Title: Selenium WebDriver - Google Search**

**Test Passed: Search results loaded successfully.**




---

**Part 5: Exploring Selenium Commands**

**Commonly Used Selenium Commands**

1. **Navigating to a URL:**

```
await driver.get('https://example.com');
```

2. **Locating Elements:**
    - **By ID:**

```
await driver.findElement(By.id('elementId'));
```

    - **By Class Name:**

```
await driver.findElement(By.className('className'));
```

    - **By Name:**

```
await driver.findElement(By.name('elementName'));
```

3. **Performing Actions:**
    - **Click an Element:**

```
await driver.findElement(By.id('buttonId')).click();
```

    - **Enter Text:**

```
await driver.findElement(By.name('inputName')).sendKeys('Hello World');
```

4. **Waiting for Elements:**

```
await driver.wait(until.elementLocated(By.id('elementId')), 5000);
```

5. **Closing the Browser:**

```
await driver.quit();
```

## 11 JavaScript Registration Form Testing using Selenium

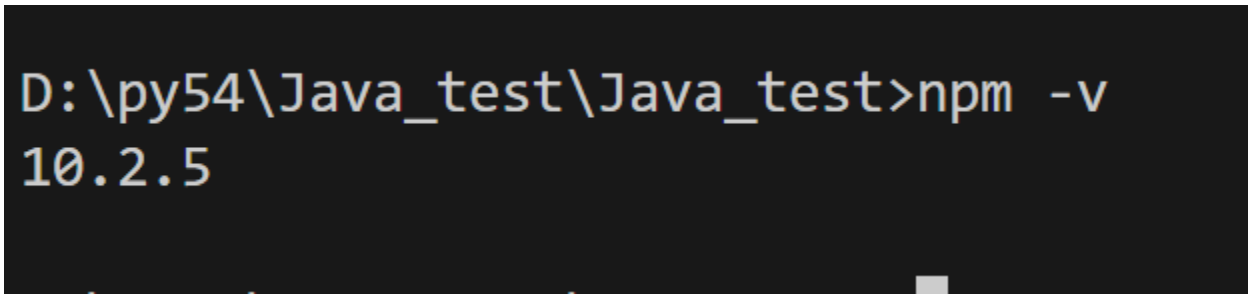
---

**Objective:**

To create a simple registration form using HTML and JavaScript, and perform automated functional testing using Selenium.

---

Firstly we have to download npm and nodejs
To check whether npm is downloaded or not use this command.




To check whether nodejs is downloaded or not use this command.

nodejs –v

And then follow below steps

**Step 1: Creating the Registration Form**

1. Open Visual Studio Code.
2. Create a folder named RegistrationForm.
3. Inside the folder, create a file named index.html with the following content:

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Registration Form</title>

<!-- Inline CSS -->

```
<style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
      background-color: #f5f5f5;
    }
    form {
      display: inline-block;
      text-align: left;
      background-color: #ffffff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    }
    input {
      width: 100%;
      padding: 10px;
      margin: 10px 0;
      box-sizing: border-box;
      border-radius: 4px;
      border: 1px solid #ccc;
    }
    button {
      background-color: #4CAF50;
color: white;
      padding: 10px 20px;
      border: none;
      border-radius: 4px;
```

```
        cursor: pointer;
    }
button:hover {
        background-color: #45a049;
    }
    #message {
color: green;
        font-weight: bold;
        margin-top: 20px;
    }
    .error {
color: red;
    }
</style>
</head>
<body>
<h2>Registration Form</h2>
<!-- Registration Form -->
<form id="registrationForm">
<label for="username">Username:</label><br>
<input type="text" id="username" name="username" placeholder="Enter username" required><br>

<label for="email">Email:</label><br>
<input type="email" id="email" name="email" placeholder="Enter email" required><br>

<label for="password">Password:</label><br>
<input type="password" id="password" name="password" placeholder="Enter password" required><br>
```

```
<button type="submit">Register</button>
</form>

<!-- Message Display -->
<p id="message"></p>

<!-- Inline JavaScript -->
<script>
    document.getElementById("registrationForm").addEventListener("submit",
function(event) {
event.preventDefault(); // Prevent form from submitting

        // Get input values
const username = document.getElementById("username").value;
const email = document.getElementById("email").value;
const password = document.getElementById("password").value;
constmessageElement = document.getElementById("message");

        // Validate inputs
        if (username === "" || email === "" || password === "") {
messageElement.textContent = "All fields are required!";
messageElement.classList.add("error");
            return;
        }

        // Check for valid email format
constemailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email)) {
messageElement.textContent = "Invalid email format!";
messageElement.classList.add("error");
```

```
            return;
        }

        // If all validations pass, show success message
    messageElement.textContent = "Registration Successful!";
    messageElement.classList.remove("error");
    messageElement.style.color = "green";

        // Clear the form
    document.getElementById("registrationForm").reset();
      });
    </script>
    </body>
    </html>
```

**Step 2: Running the Web Application**

1. Open the terminal in Visual Studio Code.
2. Use the following command to start a simple HTTP server:

```
npx http-server
```

3. Open your browser and go to http://localhost:8080 to view the form.

---

**Part 2: Testing the Registration Form using Selenium**

**Step 1: Setting up Selenium in Node.js**

1. Initialize a Node.js project:

```
npminit–y
```

```
D:\>mkdir 11th

D:\>cd 11th

D:\11th>npm init -y
Wrote to D:\11th\package.json:

{
  "name": "11th",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

2. Install the Selenium WebDriver package:

npm install selenium-webdriver

3. Download the ChromeDriver that matches your Chrome browser version from ChromeDriver Downloads.
4. Extract the downloaded chromedriver and place it in the project folder.

**Step 2: Writing the Selenium Test Script**

Create a file named test.js and add the following code:

javascript

```
const { Builder, By, Key, until } = require('selenium-webdriver');


async function testRegistrationForm() {
   // Set up WebDriver
let driver = await new Builder().forBrowser('chrome').build();
try {
      // Navigate to the registration form
awaitdriver.get('http://localhost:8080');


      // Enter username
```

```
awaitdriver.findElement(By.id('username')).sendKeys('student1');

    // Enter email
await driver.findElement(By.id('email')).sendKeys('student1@example.com');

    // Enter password
awaitdriver.findElement(By.id('password')).sendKeys('password123');

    // Click the Register button
awaitdriver.findElement(By.css('button[type="submit"]')).click();

    // Wait for the success message to appear
awaitdriver.wait(until.elementLocated(By.id('message')), 5000);

    // Get the message text
let message = await driver.findElement(By.id('message')).getText();
console.log('Test Result:', message);

    // Check if registration is successful
if (message === 'Registration Successful!') {
console.log('Test Passed');
    } else {
console.log('Test Failed');
    }
  } finally {
    // Close the browser
```

```
awaitdriver.quit();
  }
}
```

```
testRegistrationForm();
```

**Step 3: Running the Selenium Test**

1. Ensure your HTTP server is running:

```
npx http-server
```

2. Open a new terminal and run the Selenium test script:

```
node test.js
```

```
D:\11th>node Test.js

DevTools listening on ws://127.0.0.1:57433/devtools/browser/86a9542d-b92e-4d90-
a488-57ec6277b090
Test Result: Registration Successful!
Test Passed
```

---

**Expected Output:**

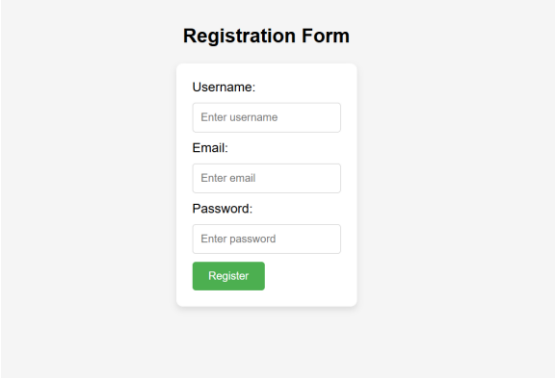
- The console should display:

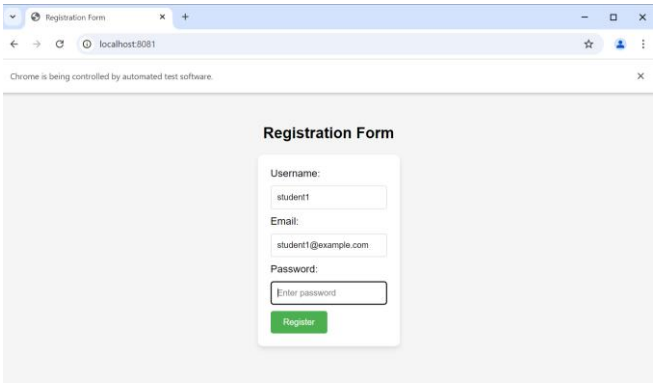
Test Result: Registration Successful!

Test Passed


```
D:\11th>node Test.js

DevTools listening on ws://127.0.0.1:57433/devtools/browser/86a9542d-b92e-4d90-
a488-57ec6277b090
Test Result: Registration Successful!
Test Passed
```


Registration Form
Username:
Enter username
Email:
Enter email
Password:
Enter password
Register


Registration Form
localhost:8081
Chrome is being controlled by automated test software.
Registration Form
Username:
student1
Email:
student1@example.com
Password:
Enter password
Register

## 12 Node.js Registration Form, Docker Containerization, and Selenium Testing

# Overview:

This project consists of a simple **Node.js registration form** hosted in a **Docker container**, with automated testing using **Selenium WebDriver**. The main steps involved are:

1. Setting up a Node.js registration form.
2. Dockerizing the application.
3. Writing Selenium tests to automate the form submission and validate the result.

### Step 1: Create a Node.js Registration Form

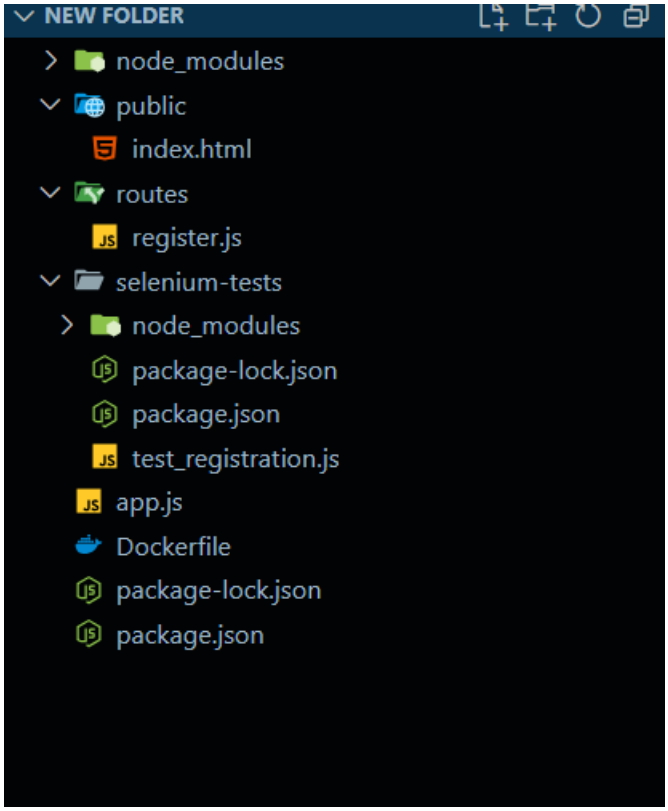
***1.1 package.json***

The package.json file defines the project metadata and dependencies. It includes necessary packages like express for the server and body-parser for handling form submissions.

**express**: A lightweight web framework for Node.js that simplifies routing and HTTP handling.

**body-parser**: Middleware to parse the incoming request bodies, enabling us to handle form submissions easily.

Project Structure




```
{
 "name": "nodejs-registration",
 "version": "1.0.0",
 "description": "Simple Node.js registration form",
 "main": "app.js",
 "scripts": {
  "start": "node app.js"
 },
 "dependencies": {
  "express": "^4.18.2",
  "body-parser": "^1.20.2"
 }
```

```
}
```

**Create a file app.js**

***1.2 app.js***

```
const express = require('express');

constbodyParser = require('body-parser');

const path = require('path');

const app = express();

// Middleware to handle form data

app.use(bodyParser.urlencoded({ extended: true }));

app.use(express.static('public'));


// Serve registration form

app.get('/', (req, res) => {

res.sendFile(path.join(__dirname, 'public', 'index.html'));

});

// Handle form submission

app.post('/register', (req, res) => {

const{ username, email, password } = req.body;

console.log(`Username: ${username}, Email: ${email}`);

res.send('Registration successful!');

});


// Start the server on port 3000

const PORT = 3000;
```

```
app.listen(PORT, () => {

console.log(`Server is running on http://localhost:${PORT}`);

});
```

**Express server setup**: Sets up an Express server that listens on port 3000.
**Static files**: Serves static files (like index.html) from the public/ folder.

**POST route**: Handles registration form submissions and logs the data to the console.

**1.3 public/index.html**

This is the HTML form where users can input their registration details (username, email, and password). Upon submission, the data is sent to the server.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>Registration Form</title>

</head>

<body>

<h2>Registration Form</h2>

<form method="POST" action="/register">

<label for="username">Username:</label>

<input type="text" id="username" name="username" required><br>

<label for="email">Email:</label>

<input type="email" id="email" name="email" required><br>

<label for="password">Password:</label>

<input type="password" id="password" name="password" required><br>

<button type="submit">Register</button>
```

</form>

</body>

</html>

### Step 2: Create a Dockerfile

A **Dockerfile** is used to containerize the Node.js application. It includes instructions to install dependencies, copy the application code, and run the app inside a container.

```
# Use Node.js base image
FROM node:14

# Set the working directory
WORKDIR /usr/src/app

# Copy package.json and install dependencies
COPYpackage.json ./
RUNnpm install

# Copy application files
COPY . .

# Expose port 3000
EXPOSE 3000

# Start the application

CMD ["npm", "start"]
```

**Base image**: We use the official node:14 Docker image as a base.
**Dependencies**: It copies the package.json file and installs the dependencies.
**Expose port**: Port 3000 is exposed, which is the port the application runs on.
**CMD**: Runs npm start to launch the app in the container

### Step 3: Build and Run Docker Container

To build and run the Docker container, execute the following commands:

# Build the Docker image

docker build -t nodejs-registration-app .

```
PS C:\Users\shiva kumar\OneDrive\Desktop\New folder> docker build -t nodejs-registration-app .
[+] Building 3.6s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 341B
 => [internal] load metadata for docker.io/library/node:14
 => [auth] library/node:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 209.40kB
 => [1/5] FROM docker.io/library/node:14@sha256:a158d3b9b4e3fa813fa6c8c590b8f0a860e015ad4e59bbce5744d2f6fd8461aa
 => CACHED [2/5] WORKDIR /usr/src/app
 => CACHED [3/5] COPY package.json ./
 => CACHED [4/5] RUN npm install
 => CACHED [5/5] COPY . .
 => exporting to image
 => => exporting layers
 => => writing image sha256:8c9e1a9f520369ce54eb60ed83ceb3ab077dc1c28349781ca88484acfe4cd6aa
 => => naming to docker.io/library/nodejs-registration-app

What's Next?
```

# Run the container

docker run -d -p 3000:3000 --name nodejs-registration nodejs-registration-app

```
What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout quickview
PS C:\Users\shiva kumar\OneDrive\Desktop\New folder> docker run -d -p 3000:3000 --name my-nodejs-app nodejs-registration-app
66c50e1726fe3d7a878564677c3aad33fc6050aa2667e4c20d83a6c35dde6a27
PS C:\Users\shiva kumar\OneDrive\Desktop\New folder>
```

After running the container, the app is accessible at http://localhost:3000.

## Step 4: Set Up Selenium for Automated Testing

To automate testing for the registration form, we use **Selenium WebDriver** to simulate a user filling out the form and submitting it.

### *4.1 Install Selenium WebDriver*

Inside the `selenium-tests` directory, initialize a new Node.js project and install `selenium-webdriver` and `chromedriver`:

By following command

Npm init -y

npm install selenium-webdriverchromedriver

### *4.2 Create test_registration.js*

This file contains the test logic for Selenium, which fills out the registration form and checks if the registration is successful.

Copy the command

```
const{ Builder, By, until } = require('selenium-webdriver');
const chrome = require('selenium-webdriver/chrome');

async function runTest() {
    let driver = await new Builder().forBrowser('chrome').build();

    try {
        // Open the registration form
        await driver.get('http://localhost:3000');

        // Fill in the registration form
        await driver.findElement(By.id('username')).sendKeys('testuser');
        await driver.findElement(By.id('email')).sendKeys('testuser@example.com');
        await driver.findElement(By.id('password')).sendKeys('password123');

        // Submit the form
        await driver.findElement(By.tagName('button')).click();

        // Wait for the response
        await driver.wait(until.elementLocated(By.tagName('body')), 5000);

        // Verify the result
        let bodyText = await driver.findElement(By.tagName('body')).getText();
```

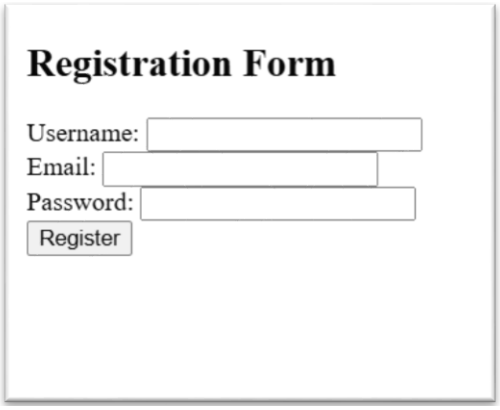
```
        if (bodyText.includes('Registration successful!')) {
console.log('Test Passed: Registration was successful.');
        } else {
console.log('Test Failed: Registration was not successful.');
        }
    } finally {
        await driver.quit();
    }
}

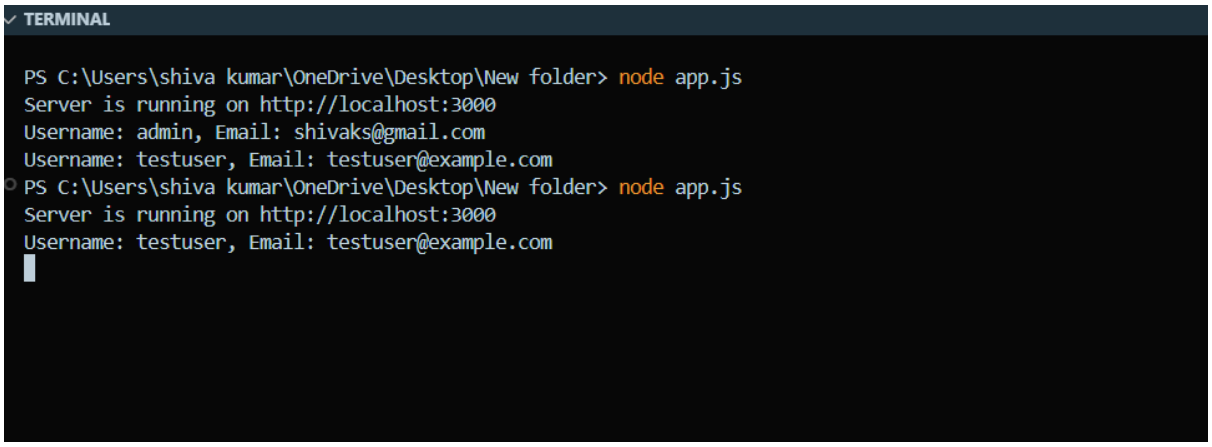
runTest();
```

**Step 5: Run Selenium Test Cases**

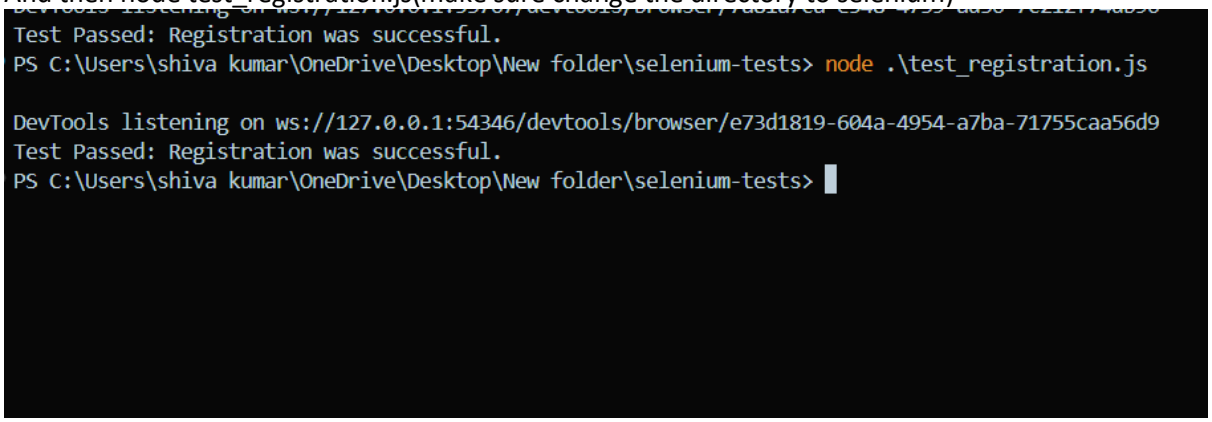
To run the automated tests, execute the following command:

First run this app.js






And then node test_registration.js(make sure change the directory to selenium)




If the registration is successful, you will see the output:

Test Passed: Registration was successful